

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

**Отчёт по практическому заданию: Очередь на двух стеках с поддержкой
минимума и анализ алгоритма Флойда**

Студент: Ошаров Александр

Группа: БПИ248

Семинарист: Маров Руслан

Дата: 2025-09-19

Содержание

1. Введение

2. Задача 1: Очередь на двух стеках с поддержкой минимума

2.1 Постановка задачи

2.2 Описание структуры и алгоритмов

2.3 Сложность алгоритмов

2.4 Тестирование

3. Задача 2: Анализ алгоритма Флойда

3.1 Доказательство встречи черепахи и зайца

3.2 Сложность $\Theta(n)$

3.3 Нахождение начала цикла

4. Выводы

5. Список литературы

1. Введение

Данный отчёт содержит реализацию структуры данных «очередь», выполненной посредством двух стеков, с поддержкой быстрого вычисления промежуточного минимума среди хранящихся целых чисел. Кроме реализации представлены тесты и анализ алгоритмической сложности. Также проведён теоретический анализ алгоритма Флойда («черепаша и заяц»): доказано, что указанные указатели встретятся, оценена сложность поиска цикла и описан метод нахождения начала цикла.

2. Задача 1: Очередь на двух стеках с поддержкой минимума

2.1 Постановка задачи

Реализовать очередь над целыми числами, используя две структуры стек, с поддержкой операции получения минимального элемента среди всех элементов очереди за $O(1)$. Обеспечить операции: `enqueue`, `dequeue`, `front`, `get_min`, `empty`.

2.2 Описание структуры и алгоритмов

Идея широко известна: храним две «стековые» структуры — `in_stack` и `out_stack`. В каждом стеке вместе с значением храним текущий минимум в этом стеке (для верхней части): то есть стек элементов пар (`value`, `current_min_in_stack`). Операция `enqueue(x)`: пушим в `in_stack` пару (`x`, `min(x, current_min_in_stack_top)`).

Операция `dequeue()`: если `out_stack` пуст — переносим все элементы из `in_stack` в `out_stack`, при этом при переносе в `out_stack` также поддерживаем `current_min` для `out_stack`. Затем просто поаем из `out_stack`.

Операция `get_min()`: если оба стека непусты — минимум равен `min(min_in_stack_top, min_out_stack_top)`, иначе — минимум одного из них.

Перенос выполняется нечасто — амортизированная сложность операций $O(1)$.

2.3 Сложность алгоритмов

Каждый элемент за время своей жизни в очереди будет перенесён из `in_stack` в `out_stack` не чаще одного раза, поэтому суммарная стоимость переносов линейна по числу операций. Таким образом амортизированная сложность операций `enqueue` и `dequeue` составляет $O(1)$. Операция получения минимума — $O(1)$ всегда.

2.4 Тестирование

Для тестовой проверки в код включены автоматические проверки (`assert`) и демонстрационная программа, которая выполняет серию операций и выводит результаты. Тесты покрывают сценарии: последовательные `enqueue`, `dequeue` до

опустошения, чередование операций и проверку корректности `get_min` в различных состояниях.

3. Задача 2: Анализ алгоритма Флойда

3.1 Доказательство встречи черепахи и зайца

Пусть последовательность узлов образует список, в котором существует цикл длины λ (>0). Пусть μ — длина участка перед началом цикла. Обозначим позицию медленного указателя (черепаха) как t , быстрого (заяц) как h . На каждом шаге черепаха сдвигается на 1, заяц на 2. Рассмотрим состояние после k шагов: $t=k$, $h=2k$ (в терминах расстояния от начала).

Если существует k такое, что $t \equiv h \pmod{\lambda}$ и $k \geq \mu$, это означает, что они находятся в одном узле цикла. Разность $2k - k = k$ должна быть кратна λ , то есть $k \equiv 0 \pmod{\lambda}$ при достижении цикл-участка. Так как числа будут расти, существует $k = \lambda \cdot s$ такой, что $k \geq \mu$ — значит произойдёт встреча.

Более формально: после μ шагов оба указателя находятся внутри цикла (за исключением случая $\mu=0$). Пусть d — расстояние внутри цикла между ними; при каждом шаге разность в позициях увеличивается на 1 (потому что заяц сокращает одну дополнительную вершину): $d \leftarrow d+1 \pmod{\lambda}$. Через некоторое число шагов разность станет 0.

3.2 Сложность $\Theta(n)$

Пусть список содержит n узлов ($n = \mu + \lambda$). На первых μ шагов хотя бы один указатель продвигается по нециклической части. В цикле, как показано выше, за $O(\lambda)$ шагов произойдёт совпадение. Следовательно общее число шагов до встречи не превосходит $\mu + \lambda = O(n)$. Каждая итерация делает $O(1)$ работы, поэтому сложность $\Theta(n)$. Более точная оценка: количество шагов $\leq \mu + \lambda$, и для нижней оценки имеется пример, где требуется $\Theta(n)$ шагов, значит $\Theta(n)$.

3.3 Нахождение точки начала цикла

После встречи (когда `hare == tortoise`), чтобы найти начало цикла, нужно разместить один указатель в начало списка, другой оставить в точке встречи. Затем двигать оба указателя по одному шагу; точка их следующей встречи будет началом цикла. Обоснование: пусть $\text{meeting_pos} = \mu + k$, где k — некоторое смещение в цикле. Если поместить один указатель в начало (позиция 0), а другой в meeting_pos , то после μ шагов первый будет в позиции μ (начало цикла), второй — в позиции $\mu + k + \mu \equiv \mu \pmod{\lambda}$. Таким образом они встретятся ровно в начале цикла.

4. Выводы

Реализация очереди на двух стеках с поддержкой получения минимума обеспечивает амортизированную сложность $O(1)$ для основных операций и постоянное время для получения минимума. Алгоритм Флойда надёжен для обнаружения цикла в последовательности указателей: его корректность и эффективность доказаны, и он позволяет также найти начало цикла.

5. Список литературы

1. Cormen T.H., Leiserson C.E., Rivest R.L., Stein C. Introduction to Algorithms.
2. CLRS, Floyd's cycle-finding algorithm (online resources).